

AI-ASSISTED CODING

LAB TEST-02

NAME: K.Manikanta

HTNO:203a52040

BATCH:03

SUB GROUP-A

QUESTION A1)

Write a Python function to parse the raw text (multiple lines) and compute per-cell averages of `latency\_ms`. Return a dict {id: avg} and separately compute an overall average

Prompt:

Implement a Python class PacketBuffer to manage values keyed by IDs.

Requirements:

- Use a dictionary for storage ({id: value}).
- Methods:
 - `add(id: str, value: float)` → add or update the value for the ID.
 - `remove(id: str)` → safely remove ID (ignore if not present).
 - `summary()` → `tuple[int, float|None]` → return (count, average) where average is rounded to 2 decimals or None if empty.
- Each operation should be $O(1)$.

CODE:

```
Welcome | latest2.py X
latest2.py > compute_per_cell_averages
1  from collections import defaultdict
2
3  def compute_per_cell_averages(raw_text):
4
5      # Initialize dictionaries for sum and count
6      cell_data = defaultdict(lambda: {'sum': 0, 'count': 0})
7      overall_sum = 0
8      overall_count = 0
9
10     # Process each line
11     for line in raw_text.strip().split('\n'):
12         try:
13             # Split the line and strip spaces
14             parts = line.strip().split(',')
15             if len(parts) != 3:
16                 continue # Skip malformed rows
17
18             cell_id, timestamp, latency = parts
19             latency = float(latency) # Convert latency to float
20
21             # Accumulate sum and count for the cell
22             cell_data[cell_id]['sum'] += latency
23             cell_data[cell_id]['count'] += 1
24
25             # Accumulate overall sum and count
26             overall_sum += latency
27             overall_count += 1
28         except (ValueError, IndexError):
29             # Skip rows with non-numeric latency or other issues
30             continue
31
```

```
Welcome | latest2.py X
latest2.py > compute_per_cell_averages
1  from collections import defaultdict
2
3  def compute_per_cell_averages(raw_text):
4
5      # Initialize dictionaries for sum and count
6      cell_data = defaultdict(lambda: {'sum': 0, 'count': 0})
7      overall_sum = 0
8      overall_count = 0
9
10     # Process each line
11     for line in raw_text.strip().split('\n'):
12         try:
13             # Split the line and strip spaces
14             parts = line.strip().split(',')
15             if len(parts) != 3:
16                 continue # Skip malformed rows
17
18             cell_id, timestamp, latency = parts
19             latency = float(latency) # Convert latency to float
20
21             # Accumulate sum and count for the cell
22             cell_data[cell_id]['sum'] += latency
23             cell_data[cell_id]['count'] += 1
24
25             # Accumulate overall sum and count
26             overall_sum += latency
27             overall_count += 1
28         except (ValueError, IndexError):
29             # Skip rows with non-numeric latency or other issues
30             continue
31
```

OUTPUT:

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS
Python: latest2 + - [ ] [ ] ... [ ] [ ] X

PS C:\AI assisted coding Lab\LAB TEST 02> & C:/Users/Mani/AppData/Local/Programs/Python/Python313/python.exe "c:/AI a
ssisted coding Lab/LAB TEST 02/latest2.py"
{'ce141': 33.7, 'ce142': 35.2, 'ce143': 36.7} and overall_avg= 35.2
PS C:\AI assisted coding Lab\LAB TEST 02> & C:/Users/Mani/AppData/Local/Programs/Python/Python313/python.exe "c:/AI a
ssisted coding Lab/LAB TEST 02/latest2.py"
{'ce141': 33.7, 'ce142': 35.2, 'ce143': 36.7} and overall_avg= 35.2
PS C:\AI assisted coding Lab\LAB TEST 02> [ ]

Ln 26, Col 35  Spaces: 4  UTF-8  CRLF  {} Python  3.13.6
```

OBSERVATON:

The program reads multiple lines of raw CSV log data where each line contains a cell ID, a timestamp, and a latency value. It processes each line, skips any malformed or invalid entries, and calculates the **average latency per cell** as well as the **overall average latency**

QUESTION –A2:

Implement a `PacketBuffer` class with methods `add(id: str, value: float)`, `remove(id: str)`, and `summary() -> tuple[int, float|None]` returning (count, average)

PROMPT:

Implement a Python class `PacketBuffer` to manage values keyed by IDs.

Requirements:

- Use a dictionary for storage (`{id: value}`).
- Methods:
 - `add(id: str, value: float) →` add or update the value for the ID.
 - `remove(id: str) →` safely remove ID (ignore if not present).
 - `summary() -> tuple[int, float|None] →` return (count, average) where average is rounded to 2 decimals or None if empty.
- Each operation should be $O(1)$.
- Example input:

CODE:

```
Welcome | latest2.py x | ▶ ▢ ...
latest2.py > ...
1  from typing import Optional, Tuple
2
3  class PacketBuffer:
4      """
5      A simple in-memory buffer to store values by ID.
6      Supports add, remove, and summary operations.
7      """
8
9      def __init__(self) -> None:
10         # Internal storage dictionary {id: value}
11         self._store: dict[str, float] = {}
12
13     def add(self, id: str, value: float) -> None:
14         """
15         Add or update a value for the given ID.
16         O(1) operation.
17         """
18         self._store[id] = value
19
20     def remove(self, id: str) -> None:
21         """
22         Safely remove the entry for the given ID.
23         Does nothing if the ID does not exist.
24         O(1) operation.
25         """
26         self._store.pop(id, None)
27
28     def summary(self) -> Tuple[int, Optional[float]]:
29         """
30         Return (count, average) of stored values.
31         - Average is rounded to 2 decimals.
32         - Returns None as average if store is empty.
```


- **summary** → return the total count of items and their average value (rounded to 2 decimals, or None if empty).